

Saveetha Institute of Medical and Technical Sciences

SAVEETHA SCHOOL OF ENGINEERING

Student Name: Moshitha Kolanjiyappan

Roll Number: 192421205

Department: Department of Information Technology

Submission Date: 10 August 2026

Assessment Title: Case Study – Autonomous Rail Navigation (Robust Feature Fitting)

1. TECHNICAL EXECUTIVE SUMMARY

Autonomous railway navigation requires reliable detection of the two steel rails from forward-looking locomotive camera images. The problem becomes challenging during heavy rain and snow because random high-frequency visual noise creates many false edges. In addition, a passing train on an adjacent track can generate another pair of nearly parallel edges, causing conventional Least Squares methods to select an incorrect rail structure.

The proposed solution combines Gaussian smoothing, Canny edge detection, geometric filtering, and RANSAC-based robust line fitting. Gaussian smoothing suppresses high-frequency noise while preserving important rail boundaries when an appropriate standard deviation (σ) is selected. Canny provides localized rail edges, while RANSAC repeatedly samples candidate points and identifies the model supported by the largest set of inliers. For a dataset containing 40% outliers, the required RANSAC iteration count for 99% confidence is approximately 11 iterations when two points are sufficient to define a line.

2. INTRODUCTION AND PROBLEM STATEMENT

Modern railway systems increasingly use computer vision to improve operational safety and situational awareness. A forward-looking camera mounted on a locomotive can continuously observe the railway track and identify unexpected objects such as fallen trees, landslides, rocks, debris, damaged infrastructure, or other obstacles.

For reliable autonomous navigation, the vision system must first understand the geometry of the railway track. The two steel rails form long, approximately parallel structures that converge toward a vanishing region in the image because of perspective. Detecting these rails provides an important geometric reference for determining the track corridor.

However, railway environments are visually complex. Rain produces bright reflections, droplets, and scattered image noise. Snow can create numerous isolated bright pixels and fragmented edge features. Fog, shadows, track-side vegetation, and moving trains can also introduce misleading edges.

A particularly difficult situation occurs when a train travels on an adjacent railway track. Its wheels, windows, body edges, and rails may generate a second set of approximately parallel line features. A conventional Least Squares line-fitting method does not distinguish between the actual rails and these unrelated features. Therefore, a robust estimation technique is required.

3. OBJECTIVES

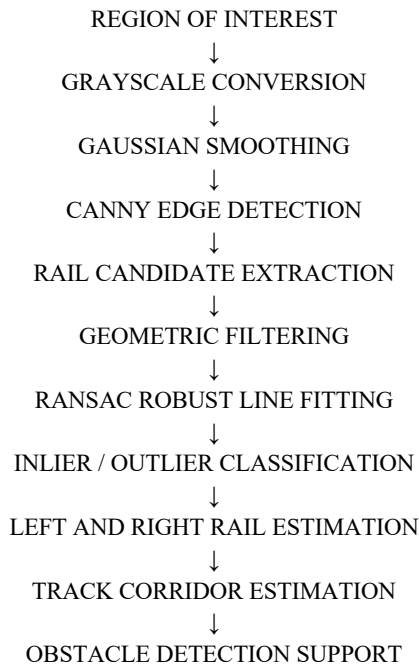
1. Reduce high-frequency rain and snow noise from railway camera images.
2. Preserve important edges corresponding to the steel rails.
3. Extract candidate rail boundaries using an edge detection technique.
4. Distinguish actual railway rails from unrelated parallel features.
5. Use RANSAC to reject outlier edge points.
6. Calculate the minimum number of RANSAC iterations required for 99% confidence.
7. Develop a computationally efficient pipeline suitable for forward-looking railway cameras.
8. Provide reliable rail geometry for subsequent obstacle-detection and navigation modules.

4. CORE ANALYSIS AND METHODOLOGY

4.1 Overall Computer Vision Pipeline

RAW CAMERA FRAME





5. QUESTION 3.1 – GAUSSIAN SMOOTHING AND σ SELECTION

5.1 Effect of Heavy Rain and Snow

Heavy rain introduces random high-frequency changes in image intensity. Water droplets, reflections, small snow particles, and sensor-level variations can appear as small bright or dark structures. Because edge detectors respond to rapid intensity changes, directly applying an edge detector to a noisy image can produce random isolated edges, broken rail boundaries, false diagonal edges, and reflections.

Gaussian smoothing is therefore applied before edge detection. The two-dimensional Gaussian function is:

$$G(x,y) = 1/(2\pi\sigma^2) \times \exp[-(x^2 + y^2)/(2\sigma^2)]$$

The smoothed image is obtained using convolution:

$$I_s(x,y) = I(x,y) * G(x,y)$$

5.2 Small σ

A small σ produces weak smoothing. Rail edges remain sharp, but rain and snow noise can remain in the image. A typical starting value may be approximately $\sigma = 0.8$ – 1.0 pixels.

- Fine rail edges remain relatively sharp.
- Rail localization is good.
- High-frequency weather noise may remain.
- More false edge points can reach the RANSAC stage.

5.3 Large σ

A large σ produces stronger smoothing. This removes more random weather noise but can blur thin and distant rail edges.

- Random rain and snow pixels are suppressed strongly.
- Small false structures can disappear.

- Weak distant rail boundaries may become blurred.
- Rail localization can become less precise.

5.4 Recommended Engineering Choice

For an initial implementation, a 5×5 Gaussian kernel with σ around 1.0–1.5 pixels is a reasonable starting point. The final value should be experimentally selected using representative railway images containing rain, snow, fog, reflections, normal daylight, and low illumination.

The correct goal is not to create the smoothest image. The goal is to produce an image in which genuine rail boundaries remain detectable while unnecessary high-frequency structures are suppressed.

6. QUESTION 3.2 – LEAST SQUARES FAILURE AND RANSAC SOLUTION

6.1 Why Least Squares Fails

A straight rail can be approximated locally by $y = mx + c$. Least Squares estimates m and c by minimizing the total squared residual:

$$E = \sum [y_i - (mx_i + c)]^2$$

Least Squares works well when most observations belong to the same underlying model. In the railway scenario, however, the image may contain actual rails, adjacent-track rails, train-body edges, wheel edges, reflections, and random snow points.

6.2 RANSAC Principle

RANSAC, or Random Sample Consensus, solves this problem by searching for a model supported by a large group of geometrically consistent observations. For a straight line, two points are sufficient to generate a candidate line.

9. Randomly select two edge points.
10. Generate a candidate line.
11. Calculate the perpendicular distance of every point from the candidate line.
12. Classify points within a selected threshold as inliers.
13. Count the number of inliers.
14. Repeat the procedure for several iterations.
15. Select the candidate model with the strongest consensus.
16. Refit the final line using the accepted inlier points.

6.3 Point-to-Line Distance

For a line $ax + by + c = 0$, the perpendicular distance from point (x_i, y_i) is:

$$d_i = |ax_i + by_i + c| / \sqrt{a^2 + b^2}$$

A point is an inlier when $d_i < \tau$, where τ is the selected RANSAC distance threshold.

6.4 How RANSAC Rejects Adjacent-Train Edges

If two randomly selected points belong to the true rail, the candidate line will be close to many actual rail points and will therefore obtain a high consensus score. If the selected points belong to an adjacent train or random snow, the resulting line will generally have fewer supporting points.

7. LEAST SQUARES VS RANSAC

Factor	Least Squares	RANSAC
Outlier handling	Poor	Strong
Multiple line structures	Can be confused	Can select strongest model
Snow/rain noise	Sensitive	Rejects inconsistent points
Adjacent train	Can distort fit	Can be rejected as outliers
Computational cost	Low	Higher
Railway suitability	Limited under severe noise	High

8. QUESTION 3.3 – RANSAC ITERATION CALCULATION

Given outlier ratio = 40%, the estimated inlier ratio is:

$$w = 1 - 0.40 = 0.60$$

For a line, the minimum sample size is two points, so $s = 2$.

The probability that one random sample consists entirely of inliers is w^s . Therefore:

$$w^s = 0.60^2 = 0.36$$

The probability that one iteration fails to choose an all-inlier sample is:

$$1 - w^s = 1 - 0.36 = 0.64$$

For 99% confidence, the allowed probability of never selecting an all-inlier sample is $1 - 0.99 = 0.01$. The standard RANSAC iteration formula is:

$$k = \log(1 - p) / \log(1 - w^s)$$

Substitute $p = 0.99$, $w = 0.60$ and $s = 2$:

$$k = \log(0.01) / \log(1 - 0.60^2)$$

$$k = \log(0.01) / \log(0.64)$$

$$k \approx 10.32$$

Since iterations must be whole numbers:

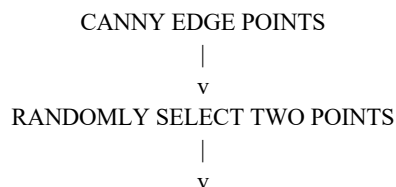
MINIMUM $k = 11$ ITERATIONS

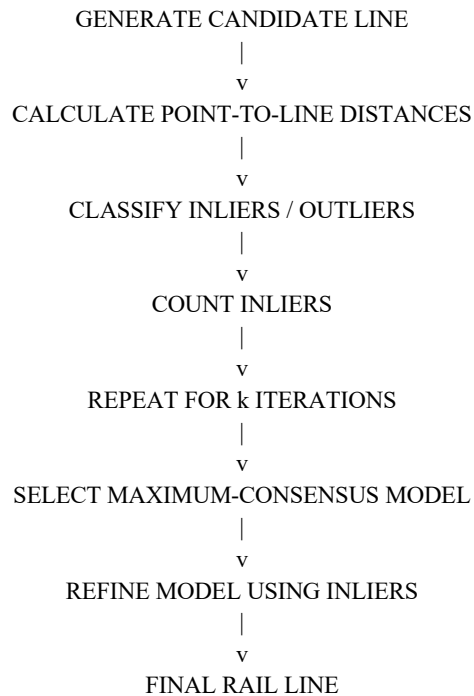
Therefore, under the stated assumptions, at least 11 RANSAC iterations are theoretically required to obtain a 99% probability of selecting at least one all-inlier sample.

9. PRACTICAL INTERPRETATION

The theoretical value of 11 iterations assumes that the estimated inlier ratio is correct, samples are independent, the distance threshold is appropriate, and the true rail model is dominant. Real railway images may violate these assumptions because edge points can be spatially clustered and rails can be partially hidden.

11. COMPLETE RANSAC WORKFLOW





12. MATHEMATICAL FORMULATION

12.1 Gaussian Filtering

$$I_s = I * G\sigma$$

12.2 Image Gradient

$$G_x = \partial I / \partial x$$

$$G_y = \partial I / \partial y$$

$$G = \sqrt{G_x^2 + G_y^2}$$

$$\theta = \tan^{-1}(G_y / G_x)$$

12.3 Rail Line Model

$$y = mx + c$$

$$\text{or: } ax + by + c = 0$$

12.4 RANSAC Residual

$$d_i = |ax_i + by_i + c| / \sqrt{a^2 + b^2}$$

12.5 RANSAC Iteration Formula

$$k = \log(1 - p) / \log(1 - w^s)$$

13. ENGINEERING TRADE-OFFS AND LIMITATIONS

13.1 Gaussian Smoothing

Gaussian smoothing improves robustness to high-frequency weather noise, but excessive smoothing can remove thin rail boundaries. The engineering solution is to use a moderate σ and validate it on representative weather conditions.

13.2 Canny versus Sobel

Canny is preferred over a simple Sobel gradient when a clean edge map is required because it includes smoothing, gradient calculation, non-maximum suppression, and hysteresis-based edge linking. Sobel is simpler and computationally cheaper but can produce thicker and more noisy responses.

13.3 Least Squares versus RANSAC

Least Squares is computationally efficient but highly sensitive to outliers. RANSAC requires repeated model generation and testing, which increases computational cost. However, this extra cost is justified when the image contains substantial false edges and multiple line structures.

14. HANDLING WEATHER AND ENVIRONMENTAL CHANGES

Condition	Proposed Handling
Heavy rain	Gaussian smoothing + Canny + RANSAC
Snow	Noise suppression followed by robust geometric fitting
Fog	Contrast-aware preprocessing and adaptive thresholds
Night	Exposure-aware preprocessing and threshold adjustment
Adjacent train	Region constraints + RANSAC + geometry
Partial rail occlusion	RANSAC fitting using visible inlier sections

15. TEMPORAL CONSISTENCY

A practical railway system should process video rather than treating every frame independently. The previous frame's rail estimates can be compared with the current frame. If the current estimate suddenly changes beyond a physically reasonable amount, the system can reduce confidence and use temporal filtering.

16. PSEUDOCODE

START

Capture forward-facing railway image
Select railway region of interest
Convert image to grayscale
Apply Gaussian smoothing
Apply Canny edge detection
Extract candidate edge points
Apply geometric constraints

Initialize best_model = NULL
Initialize best_inlier_count = 0

FOR each RANSAC iteration:

 Randomly select two candidate points

```

Construct candidate line

FOR every candidate point:
    Calculate point-to-line distance

    IF distance < threshold:
        classify as INLIER
    ELSE:
        classify as OUTLIER

Count number of inliers

IF current inlier count > best_inlier_count:
    Save current model
    Update best_inlier_count
END FOR

Refine best rail model using inliers
Repeat for second rail
Check rail geometry
Estimate track corridor
Send rail geometry to obstacle detection
END

```

17. PYTHON / OPENCV IMPLEMENTATION CONCEPT

```

import cv2
import numpy as np

# Read forward-facing camera frame
image = cv2.imread("railway_frame.jpg")

# Convert to grayscale
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Gaussian smoothing
blur = cv2.GaussianBlur(
    gray,
    (5, 5),
    1.2
)

# Canny edge detection
edges = cv2.Canny(
    blur,
    50,
    150
)

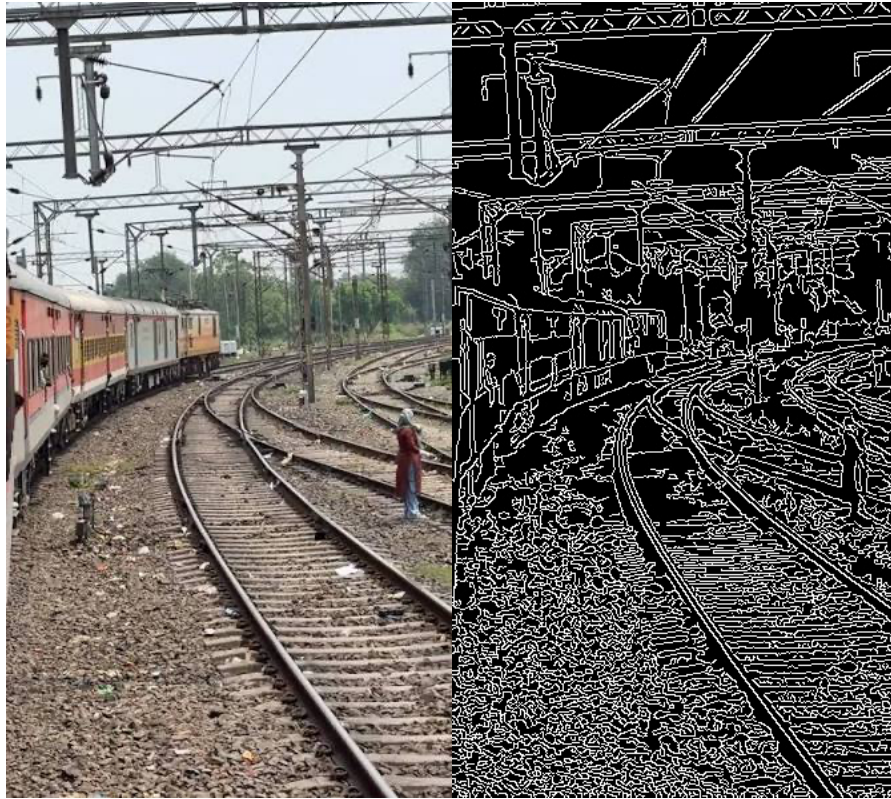
# Extract edge coordinates
points = np.column_stack(np.where(edges > 0))

# RANSAC line fitting is applied
# to the candidate rail points.

cv2.imwrite("rail_edges.jpg", edges)

```

The code represents the preprocessing portion of the proposed pipeline. A complete railway implementation would additionally apply region-of-interest constraints, geometric filtering, RANSAC fitting, temporal tracking, and safety validation.



18. VERIFICATION AND VALIDATION PLAN

The system should be evaluated under multiple realistic conditions rather than only clear-weather images.

- Normal daylight.
- Heavy rain.
- Light rain.
- Snowfall.
- Foggy conditions.
- Low-light conditions.

18.1 Evaluation Metrics

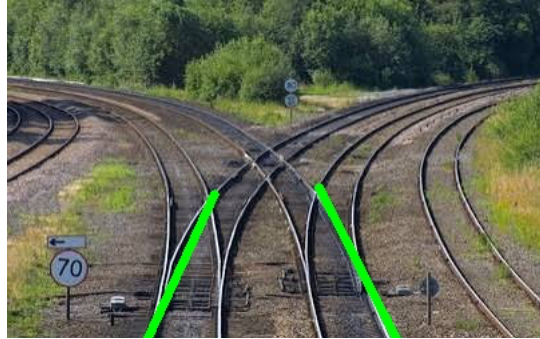
- Rail Detection Accuracy – percentage of frames in which both rails are correctly detected.
- Line Fitting Error – distance between predicted rail lines and manually annotated ground-truth points.
- Outlier Rejection Rate – proportion of irrelevant points successfully rejected.
- Processing Time – average time required per frame.

19. EXPECTED RESULTS

The expected result is a system that produces two stable rail-line estimates even when the image contains substantial visual noise. Under normal conditions, Canny should generate clear rail boundaries. During heavy rain or snow, Gaussian smoothing should reduce random high-frequency responses before edge extraction.

When an adjacent train introduces another parallel structure, Least Squares may incorrectly fit the combined point distribution. RANSAC should instead identify the model supported by the strongest geometrically consistent group of rail points.

For the specified 40% outlier condition, the theoretical calculation gives approximately 11 iterations for 99% confidence, assuming a two-point line model and an inlier ratio of 60%.



20. LIMITATIONS

- Extremely heavy snow may completely hide the rail.
- Severe fog can reduce edge contrast.
- RANSAC requires more computation than ordinary Least Squares.
- A moving adjacent train may generate a large number of structured features.
- Perspective distortion means that rails are not perfectly represented by a single simple image-space line across the whole image.
- Camera vibration and lens contamination can further degrade detection.

21. ENGINEERING IMPROVEMENTS

- Adaptive Gaussian filtering based on estimated image noise.
- Adaptive Canny thresholds based on image statistics.
- Separate RANSAC processing for left and right track regions.
- Vanishing-point constraints to reject unrealistic line orientations.
- Parallelism constraints between left and right rails.
- Temporal tracking across consecutive video frames.
- Confidence scoring based on inlier count, residual error, geometry, and temporal consistency.

22. CONCLUSION

Autonomous railway navigation requires robust computer vision because real railway images contain noise, reflections, weather effects, occlusions, and multiple competing line structures. A simple Least Squares regression model is not sufficiently reliable because it assumes that the observed points are primarily generated by one model.

Gaussian smoothing provides the first level of robustness by suppressing high-frequency rain and snow noise. Canny edge detection then extracts candidate rail boundaries. RANSAC provides the critical robust-fitting stage by repeatedly sampling candidate points and selecting the model supported by the largest number of geometrically consistent inliers. This allows unrelated edges from an adjacent train, falling snow, and other visual structures to be rejected. A larger configurable iteration limit and additional geometric and temporal constraints should be used.

Overall, the proposed Gaussian smoothing + Canny + RANSAC pipeline provides a practical foundation for robust rail estimation and can serve as an important preprocessing stage for autonomous railway obstacle detection.

23. REFERENCES

1. OpenCV Documentation, GaussianBlur – Image Filtering.
https://docs.opencv.org/4.5.5/d4/d86/group__imgproc__filter.html
2. OpenCV Documentation, Canny Edge Detection.
https://docs.opencv.org/4.12.0/da/d22/tutorial_py_canny.html
3. OpenCV Documentation, Camera Calibration and 3D Reconstruction / RANSAC.
https://docs.opencv.org/3.3.1/d9/d0c/group__calib3d.html
4. OpenCV Documentation, Features2D + Homography.
https://docs.opencv.org/4.13.0/d7/dff/tutorial_feature_homography.html
5. Fischler, M. A., & Bolles, R. C. (1981). Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography. Communications of the ACM.

24. FINAL ENGINEERING RESULT

The proposed autonomous rail-navigation vision system combines Gaussian smoothing, Canny edge detection, geometric filtering, and RANSAC robust line fitting to detect railway rails under severe visual noise. Gaussian smoothing controls high-frequency rain and snow noise, while Canny provides localized edge information. RANSAC prevents unrelated parallel structures, including adjacent trains, from corrupting the rail model.

For 40% outliers, 60% inliers, a two-point line model, and 99% confidence:

$$k = \log(1 - 0.99) / \log(1 - 0.60^2) \approx 10.32$$

Therefore, the theoretical minimum is:

MINIMUM RANSAC ITERATIONS = 11